Введение в Express.js

Создание серверного приложения

Результаты занятия

- ✓ Объяснять назначение Express.js и его место в backend-разработке
- ✓ Создавать Node.js-проект для серверного приложения
- ✓ Устанавливать и подключать Express
- ✓ Создавать и запускать HTTP-сервер
- ✓ Создавать простые GET-маршруты
- ✓ Использовать объекты req и res
- ✓ Возвращать текстовые и JSON-ответы клиенту

Учебные вопросы

Назначение Express.js
Роль Express в Node.js и fullstack-приложении

Создание Express-проекта
Инициализация проекта, package.json, ES Modules и установка зависимостей

Создание и запуск сервера
Объект приложения, порт, app.listen() и запуск процесса Node.js

Маршруты, Request и Response

Обработка HTTP-запросов и формирование ответов

Первое API на Express
JSON, несколько маршрутов и проверка работы приложения

Учебный вопрос 1. Назначение Express.js

Зачем нужен Express, если Node.js уже умеет работать с HTTP

От frontend к backend

- Frontend работает в браузере и отвечает за пользовательский интерфейс.
- Backend работает на сервере и принимает запросы от клиентского приложения.
- Backend выполняет бизнес-логику, обращается к базе данных и проверяет права пользователя.
- Frontend и backend взаимодействуют через HTTP-запросы и HTTP-ответы.
- В нашем курсе серверная часть будет разрабатываться на Node.js и Express.js.

Главная идея

Express станет основой REST API будущего fullstack-приложения.

Что уже предоставляет Node.js

- Node.js позволяет выполнять JavaScript вне браузера.
- Встроенный модуль http позволяет открыть сетевой порт и принимать HTTP-запросы.
- Разработчик может самостоятельно анализировать URL и HTTP-метод запроса.
- Можно вручную устанавливать статус ответа, заголовки и отправлять данные клиенту.
- Следовательно, Express не является обязательным условием создания HTTP-сервера.

Главная идея

Проблема возникает не в возможности создать сервер, а в удобстве разработки и сопровождения приложения.

Сервер на чистом Node.js и на Express

Node.js http

- Низкоуровневая работа с запросом и ответом
- Ручная проверка URL и HTTP-методов
- Ручная организация маршрутизации
- Больше технического кода
- Полезен для понимания устройства HTTP-сервера

Express.js

- Удобный API поверх возможностей Node.js
- Готовая система маршрутизации
- Удобная работа с req и res
- Поддержка цепочек middleware
- Меньше служебного кода в прикладном приложении

Node.js и Express.js: уровни серверного приложения

BUILD
LEARN
DEVELOP
GROW

Каждый уровень использует возможности нижележащего уровня

Маршруты и серверная логика приложения

GET /products | POST /orders | валидация | бизнес-логика | REST API

ВАШЕ ПРИЛОЖЕНИЕ

Express.js

framework | routing | middleware | req/res

Express.js не заменяет Node.js, а работает поверх его возможностей

Node.js и HTTP API

Среда выполнения JavaScript | http | net | fs | process

RUNTIME И НИЗКОУРОВНЕВЫЕ ВОЗМОЖНОСТИ

Операционная система и сеть

Сервер (оборудование) | ОС (Linux / Windows / macOS) | Сеть | TCP/IP | Порт (например, 3000) | Файлы и сетевая система

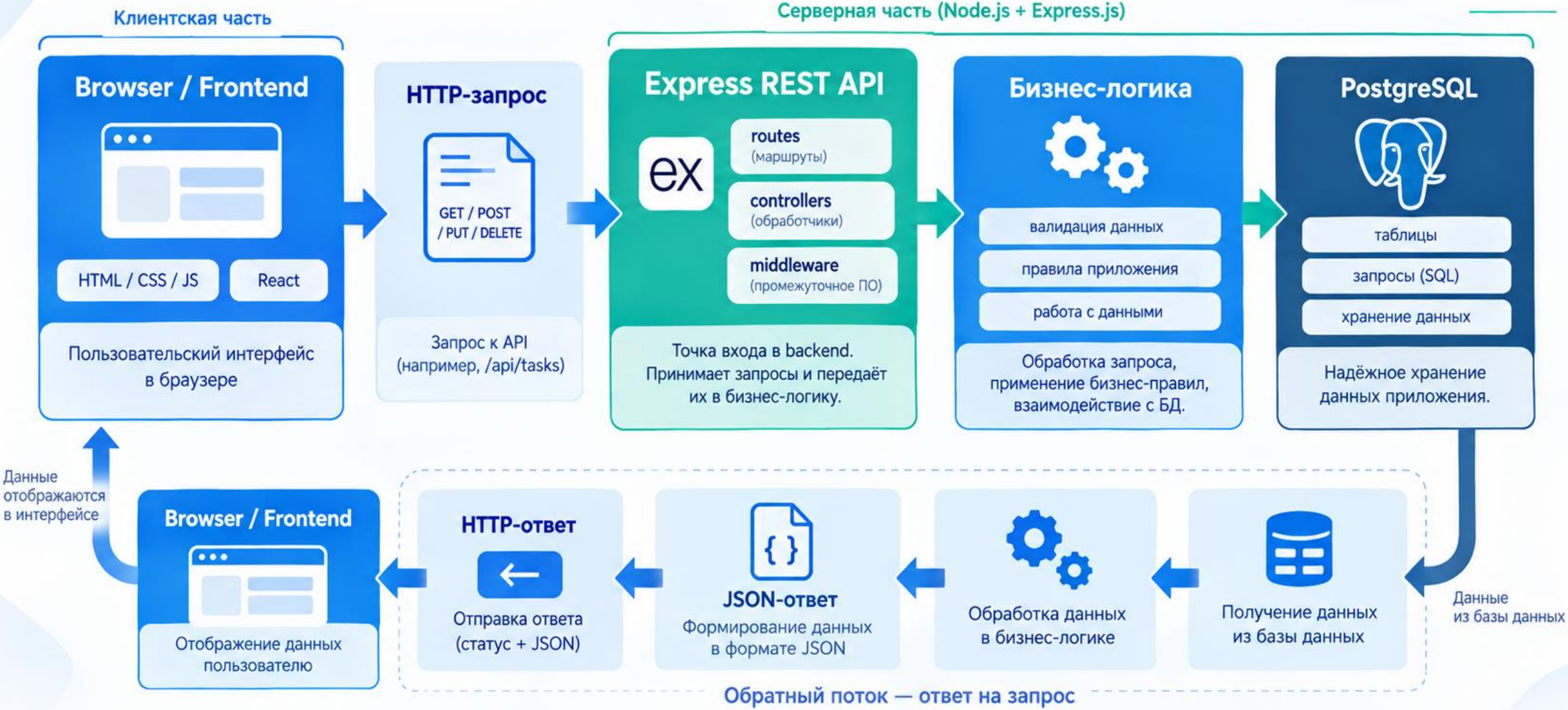
ИНФРАСТРУКТУРНАЯ ОСНОВА

Что такое Express.js

- Express.js — минималистичный веб-фреймворк для Node.js, предоставляющий средства маршрутизации, обработки HTTP-запросов и построения серверных приложений.
- Express работает внутри процесса Node.js.
- Приложение Express остаётся обычным JavaScript-приложением.
- Express использует стандартные объекты HTTP-запроса и ответа Node.js.
- Фреймворк добавляет удобные методы и механизм middleware.
- Express часто используется для создания REST API.

Место Express.js в fullstack-приложении

УЧИСЬ
СОЗДАВАЙ
РАЗВИВАЙСЯ
В IT



Основные задачи Express

Routing

Определяет, какой код выполнить для конкретного HTTP-метода и URL.

Request

Предоставляет удобный доступ к данным входящего HTTP-запроса.

Response

Помогает сформировать текстовый, JSON или другой HTTP-ответ.

Middleware

Позволяет выполнять дополнительную обработку запроса до получения ответа.

Express — это не готовое приложение

Express предоставляет

- Механизм маршрутизации
- Работу с HTTP-запросами
- Работу с HTTP-ответами
- Систему middleware
- Базу для построения REST API

Разработчик проектирует сам

- Структуру базы данных
- Бизнес-логику
- Модель пользователей и ролей
- Структуру проекта
- Правила обработки данных

Учебный вопрос 2. Создание Express-проекта

Подготовка каталога, package.json, модульной системы и зависимостей

Что необходимо для работы

- ✓ Node.js — среда выполнения серверного JavaScript
- ✓ npm — менеджер пакетов, устанавливаемый вместе с Node.js
- ✓ Visual Studio Code — редактор исходного кода
- ✓ Терминал PowerShell, Command Prompt или встроенный терминал VS Code
- ✓ Браузер или HTTP-клиент для проверки запросов
- ✓ Отдельная папка для проекта

Шаг 1. Создаём каталог проекта

1. **Создать рабочую папку**

   Каждое Node.js-приложение удобно хранить в отдельном каталоге.
   mkdir express-intro

2. **Перейти в каталог**

   Последующие команды npm будут выполняться относительно текущей папки.
   cd express-intro

3. **Открыть каталог в VS Code**

   Команда доступна, если VS Code добавлен в PATH.
   code .

Шаг 2. Инициализируем Node.js-проект

1	**Выполнить npm init**	Команда создаёт описание Node.js-проекта. npm init -y
2	**Проверить package.json**	Файл появится в корне текущего каталога.
3	**Использовать проект как npm-проект**	Теперь npm сможет сохранять зависимости и команды запуска приложения.

Что находится в package.json

```
{
  "name": "express-intro",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {},
  "license": "ISC"
}
```

Что важно

- name — имя проекта в экосистеме npm.
- version — версия приложения или пакета.
- main — исторически используемая точка входа пакета; запуск приложения мы настроим отдельно.
- scripts — команды, которые можно запускать через npm.
- После установки Express появится раздел dependencies.

Шаг 3. Включаем ES Modules

```
{
  "name": "express-intro",
  "version": "1.0.0",
  "type": "module"
}
```

Что важно

- Поле "type": "module" сообщает Node.js, что файлы .js проекта используют систему ECMAScript Modules.
- После этого библиотеки можно подключать через синтаксис import.
- В курсе будем использовать современную запись import и export.
- Без этой настройки аналогичный проект часто использует CommonJS и функцию require().

Шаг 4. Устанавливаем Express

```
npm install express
```

Что важно

- npm скачивает Express и необходимые ему зависимости.
- В проекте создаётся каталог node_modules.
- В package.json Express записывается в раздел dependencies.
- package-lock.json фиксирует конкретное дерево установленных зависимостей.
- После установки Express можно импортировать в исходный код приложения.

Файлы после установки Express

Элемент	Назначение	Работа разработчика
package.json	Описание проекта и список зависимостей	Редактируется
package-lock.json	Фиксация установленных версий	Обычно вручную не редактируется
node_modules	Файлы установленных npm-пакетов	Вручную не редактируются
app.js	Исходный код нашего сервера	Создаётся разработчиком

Структура первого проекта

```
express-intro/
├── node_modules/
├── app.js
├── package.json
└── package-lock.json
```

Что важно

- На первом занятии всё приложение можно разместить в одном файле app.js.
- Так проще увидеть полный жизненный цикл небольшого сервера.
- Позже маршруты, контроллеры и сервисы будут разделены по отдельным файлам.
- Каталог node_modules не следует вручную переносить между компьютерами — зависимости восстанавливаются командой npm install.

Учебный вопрос 3. Создание и запуск сервера

Импорт Express, объект app, порт и прослушивание соединений

Первое Express-приложение

```
import express from "express";

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("Hello Express!");
});

app.listen(PORT, () => {
  console.log(`Server: http://localhost:${PORT}`);
});
```

1. Подключаем Express

```
import express from "express";

const app = express();
```

Что важно

- Первая строка импортирует установленный npm-пакет express.
- Вызов express() создаёт экземпляр Express-приложения.
- Переменная app становится центральным объектом серверной части.
- Через app мы будем подключать маршруты, middleware и запускать сервер.

2. Выбираем порт

```
const PORT = 3000;
```

Что важно

- HTTP-сервер должен прослушивать определённый TCP-порт.
- В учебных Node.js-приложениях часто используют порт 3000.
- При локальной разработке адрес будет иметь вид http://localhost:3000.
- localhost обозначает текущий компьютер, на котором запущено приложение.
- Позже номер порта будем получать из переменных окружения.

3. Запускаем сервер

```
app.listen(PORT, () => {
  console.log(`Server: http://localhost:${PORT}`);
});
```

Что важно

- app.listen() запускает HTTP-сервер.
- Сервер начинает ожидать входящие соединения на указанном порту.
- Функция обратного вызова выполняется после успешного запуска прослушивания.
- После запуска процесс Node.js не завершается, потому что продолжает ожидать новые запросы.

Запуск приложения из терминала

1	**Открыть терминал в каталоге проекта**	Текущей папкой должен быть каталог, содержащий app.js.
2	**Запустить Node.js**	Node.js загрузит app.js и выполнит код приложения. node app.js
3	**Проверить сообщение**	В терминале должен появиться адрес http://localhost:3000.
4	**Открыть адрес в браузере**	Браузер отправит HTTP-запрос серверу.
5	**Остановить сервер**	В терминале процесс можно завершить сочетанием Ctrl+C.

Что происходит после запуска app.listen()

УЧИСЬ СОЗДАВАЙ РАЗВИВАЙСЯ В IT

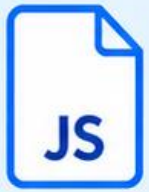
1 node app.js



```
node app.js
```

Выполнение команды в терминале

2 Загрузка JavaScript



Node.js загружает файл и начинает выполнять JavaScript-код сверху вниз

3 Создание объекта Express

ex

```
const app = express()
```

Создаётся экземпляр приложения Express

4 Регистрация маршрутов

</>

```
app.get('/', ...)
app.get('/api/status', ...)
```

Настраиваются маршруты и обработчики запросов

5 Вызов app.listen(3000)



```
app.listen(3000)
```

Вызывается метод для запуска сервера

6 Открытие порта 3000



Node.js начинает слушать порт 3000 и принимает входящие соединения

7 Процесс Node.js остаётся запущенным

node JS

Процесс не завершается после выполнения файла. Он продолжает работать и ждать запросы.

8 Сервер ожидает запросы



Клиенты (браузер или другие приложения) отправляют HTTP-запросы, а сервер их обрабатывает.

Последовательность событий после запуска файла

Сервер не завершает работу после выполнения файла

Вызов `app.listen()` запускает HTTP-сервер, который продолжает работать в фоновом режиме. Даже когда весь код в файле выполнен, процесс Node.js остаётся активным, чтобы обрабатывать входящие запросы.

- ✓ Файл выполнился
- ✓ Сервер запущен
- ✓ Процесс не завершается
- ✓ Сервер ждёт запросы

Учебный вопрос 4. Маршруты, Request и Response

Как Express понимает, какой код выполнить для HTTP-запроса

Первый маршрут

```
app.get("/", (req, res) => {
  res.send("Hello Express!");
});
```

Что важно

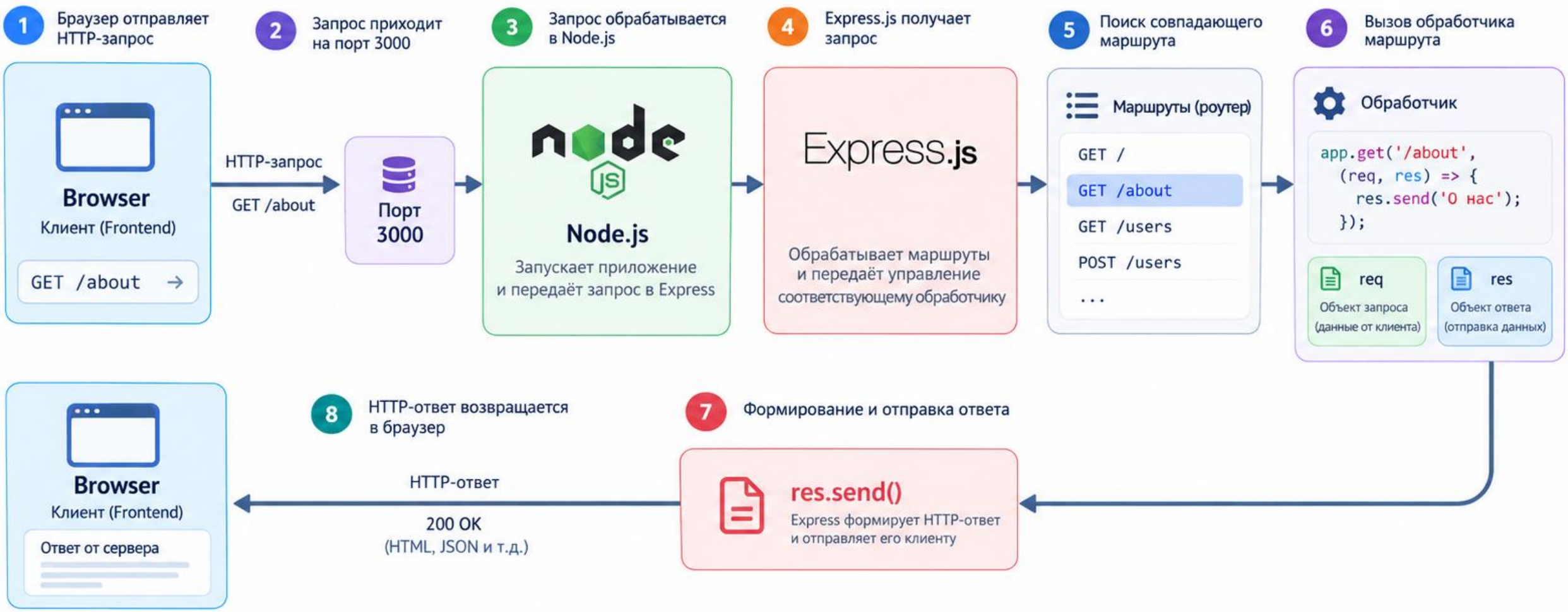
- app.get() регистрирует обработчик HTTP-запроса методом GET.
- Строка "/" обозначает корневой URL приложения.
- Функция (req, res) => {} выполняется, когда запрос соответствует этому маршруту.
- req представляет входящий запрос клиента.
- res используется для формирования и отправки HTTP-ответа.

Express.js
Как это работает

Путь HTTP-запроса через Express

От браузера до обработчика и обратно

Один запрос —
много шагов,
но всё работает вместе



Весь этот путь происходит автоматически при каждом запросе!

Express.js упрощает работу с HTTP, но использует возможности Node.js

Ключевая идея:

Express принимает запрос, находит нужный маршрут и передаёт управление вашему обработчику, а вы используете req и res для работы с данными.

Объекты req и res

req — Request

Представляет входящий HTTP-запрос. Через него приложение получает сведения о URL, параметрах, заголовках и переданных клиентом данных.

res — Response

Представляет HTTP-ответ. Через него приложение задаёт статус и отправляет клиенту текст, JSON или другое содержимое.

Отправляем текстовый ответ

```
app.get("/about", (req, res) => {
  res.send("Учебный интернет-магазин");
});
```

Что важно

- Клиент отправляет GET-запрос на адрес /about.
- Express находит обработчик с подходящими HTTP-методом и путём.
- res.send() формирует и отправляет ответ.
- После отправки ответа клиент может отобразить полученное содержимое.

Несколько URL — несколько обработчиков

Пример

```
app.get("/", (req, res) => {
  res.send("Главная");
});

app.get("/about", (req, res) => {
  res.send("О проекте");
});

app.get("/contacts", (req, res) => {
  res.send("Контакты");
});
```

Разбор

- Express сопоставляет входящий запрос с зарегистрированным маршрутом. Поэтому разные URL могут запускать разные функции-обработчики.

Из чего состоит маршрут

Элемент	Пример	Назначение
HTTP-метод	GET	Определяет тип операции
Путь	/about	Определяет адрес ресурса
Обработчик	(req, res) => {}	Содержит выполняемый код
Ответ	res.send(...)	Возвращает результат клиенту

Учебный вопрос 5. Первое API на Express

Передаём структурированные данные между backend и frontend

Почему серверу нужен JSON

- Frontend обычно должен получать не готовый текст страницы, а структурированные данные.
- JavaScript в браузере умеет преобразовывать JSON в обычные объекты и массивы.
- JSON удобно передавать через HTTP.
- Объекты товаров, пользователей и заказов можно представить в JSON.
- REST API нашего интернет-магазина в основном будет возвращать JSON.

Главная идея

Backend предоставляет данные, а frontend решает, как показать их пользователю.

Возвращаем JSON-объект

```
app.get("/api/status", (req, res) => {
  res.json({
    status: "ok",
    message: "Server is running"
  });
});
```

Что важно

- res.json() принимает JavaScript-объект или массив.
- Express сериализует значение в JSON и отправляет клиенту.
- Такой ответ легко обрабатывать во frontend-приложении.
- Префикс /api часто используют для маршрутов программного API.

Возвращаем массив товаров

Пример

```
const products = [
  { id: 1, name: "Keyboard", price: 15000 },
  { id: 2, name: "Mouse", price: 8000 }
];

app.get("/api/products", (req, res) => {
  res.json(products);
});
```

Разбор

- Пока данные находятся в памяти приложения. В следующих разделах тот же маршрут сможет получать товары из PostgreSQL.

res.send() и res.json()

res.send()

- Универсальная отправка содержимого
- Удобен для простого текста
- Подходит для небольших учебных ответов
- Пример: res.send("Hello")

res.json()

- Ориентирован на JSON-ответ
- Удобен для объектов и массивов
- Основной вариант для REST API
- Пример: res.json({ status: "ok" })

Статус HTTP-ответа

Статус	Название	Когда встречается
200	OK	Запрос успешно обработан
201	Created	Ресурс успешно создан
400	Bad Request	Клиент передал некорректные данные
404	Not Found	Маршрут или ресурс не найден
500	Internal Server Error	Ошибка на стороне сервера

Явно задаём HTTP-статус

```
app.get("/api/status", (req, res) => {
  res.status(200).json({
    status: "ok"
  });
});
```

Что важно

- res.status() задаёт HTTP-код ответа.
- После status() можно продолжить цепочку вызовов и вызвать json().
- Код 200 означает успешное выполнение запроса.
- Корректные статусы позволяют frontend отличать успешный ответ от ошибки.

Первое знакомство с middleware

```
app.use(express.json());
```

Что важно

- Middleware — функция, участвующая в обработке HTTP-запроса между моментом его получения сервером и формированием окончательного ответа.
- express.json() — встроенный middleware Express.
- Он предназначен для разбора JSON в теле входящих запросов.
- В сегодняшнем приложении с GET-маршрутами он ещё почти не нужен.
- При работе с POST и PATCH запросами значение станет доступно через req.body.
- Middleware подробно изучим в отдельной лекции.

Первое небольшое API целиком

```
import express from "express";

const app = express();
const PORT = 3000;

app.use(express.json());

app.get("/", (req, res) => {
  res.send("Internet Shop API");
});

app.get("/api/status", (req, res) => {
  res.json({ status: "ok" });
});

app.listen(PORT, () => {
  console.log(`Server: http://localhost:${PORT}`);
});
```

Express.js
Разработка шаг за шагом

Цикл разработки и проверки Express-приложения

Изменяй код → запускай → проверяй → улучшай → повторяй

Разработка — это повторяющийся процесс

1 Написать или изменить код

Редактируем файлы в редакторе

```
app.get('/about', (req, res) => {
  res.send('О проекте');
});
```

Например, добавляем новый маршрут

2 Запустить сервер

Выполняем приложение в терминале

```
PS C:\express-shop> npm start

> express-shop@1.0.0 start
> node app.js

Server: http://localhost:3000
```

Сервер должен запуститься без ошибок

3 Отправить запрос

Проверяем в браузере или HTTP-клиенте



4 Получить ответ

Проверяем результат

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

О проекте
```

- Правильный статус
- Ожидаемые данные
- JSON или текст

5 Проверить терминал и результат

Анализируем логи и поведение приложения

```
GET /about
Server: http://localhost:3000
```

- Нет ошибок
- Запрос обработан
- Логи соответствуют ожиданиям

6 Исправить код при необходимости

Если результат не такой, как ожидалось

```
// Исправляем текст
app.get('/about', (req, res) => {
  res.send('Учебный проект');
});
```

Вносим изменения в код и сохраняем файл

7 Повторить проверку

Снова запускаем, отправляем запрос и убеждаемся, что всё работает

Повторяем цикл, пока результат не будет корректным

Express.js

Пишем. Запускаем.
Проверяем. Улучшаем.

Главная идея: небольшие изменения → быстрая проверка → стабильный результат

Так мы постепенно развиваем наше Express-приложение.

Совет

Используйте браузер для простых GET-запросов и HTTP-клиент (например, Thunder Client) — для более сложных запросов.

Мини-практика

Дополните сервер собственными маршрутами и проверьте их работу.

Задание

- GET / — вернуть название приложения
- GET /about — вернуть описание проекта
- GET /api/status — вернуть JSON со статусом сервера
- GET /api/products — вернуть JSON-массив минимум из двух товаров

Критерии проверки

- Приложение запускается без ошибок
- Каждый URL вызывает свой обработчик
- Текстовые маршруты используют res.send()
- API-маршруты возвращают корректный JSON

Контрольные вопросы

1. Какую роль Express.js выполняет в Node.js-приложении?
2. Почему HTTP-сервер можно создать без Express.js?
3. Для чего выполняется команда npm init -y?
4. Что хранится в файле package.json?
5. Для чего в package.json используется поле "type": "module"?
6. Что создаёт вызов express()?
7. Для чего используется метод app.listen()?
8. Из каких основных частей состоит маршрут Express?
9. Чем отличаются объекты req и res?
10. Когда целесообразно использовать res.json()?

Итоги занятия

1. Express.js работает поверх Node.js и упрощает создание серверных приложений.
2. Express устанавливается как зависимость npm-проекта.
3. Объект app является центральным объектом Express-приложения.
4. app.listen() запускает HTTP-сервер и открывает порт для входящих соединений.
5. Маршрут связывает HTTP-метод и URL с функцией-обработчиком.
6. req представляет запрос клиента, а res используется для формирования ответа.
7. REST API обычно передаёт frontend структурированные данные в формате JSON.